

1^η ΣΑΕΚ ΗΡΑΚΛΕΙΟΥ

ΜΑΘΗΜΑ: ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΑΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ (C++)

Φύλλο Ανάθεσης Εργασίας Αριθμ. 5

Συνέχεια στις Συναρτήσεις

Δραστηριότητα 1 – Αναδρομική κλήση συνάρτησης

Δημιουργήστε ένα αρχείο με όνομα factorial_recursive.cpp με το παρακάτω περιεχόμενο και εκτελέστε το:

```
#include<iostream>
using namespace std;

int factorial(int n)
{
    if(n > 1)
        return n * factorial(n - 1);
    else
        return 1;
}

int main()
{
    int n;

    cout << "Enter a positive integer: ";
    cin >> n;

    cout << "Factorial of " << n << " = " << factorial(n);

    return 0;
}
```

Δραστηριότητα 2 – Υπολογισμός Παραγοντικού

Να γράψετε συνάρτηση PARAG που να δέχεται σαν είσοδο ένα αριθμό N (όπου $N \geq 1$) και να επιστρέφει το παραγοντικό του ($N!$) δηλαδή το $1*2*3*...*N$. Ο υπολογισμός του παραγοντικού να γίνει με επανάληψη και όχι αναδρομικά.

Στη συνέχεια χρησιμοποιώντας την παραπάνω συνάρτηση να υπολογίσετε την παρακάτω παράσταση:
 $A=49!/(6!*43!)$ και την παράσταση $B=20*45!/(5!*40!)$

Δραστηριότητα 3 – Υπερφόρτωση συνάρτησης

Δημιουργήστε ένα αρχείο με όνομα `overload.cpp` με το παρακάτω περιεχόμενο και εκτελέστε το:

```
// overloading functions
#include <iostream>
using namespace std;

int operate (int a, int b)
{
    return (a*b);
}

double operate (double a, double b)
{
    return (a/b);
}

int main ()
{
    int x=5,y=2;
    double n=5.0,m=2.0;
    cout << operate (x,y) << '\n';
    cout << operate (n,m) << '\n';
    return 0;
}
```

Δραστηριότητα 4 – Άσκηση με Υπερφόρτωση συνάρτησης

Θα δημιουργήσετε μια συνάρτηση που να ονομάζεται `square` και να υπολογίζει το τετράγωνο ενός αριθμού. Το τετράγωνο να υπολογίζεται με απλό πολλαπλασιασμό.

Συγκεκριμένα, να υπερφορτώσετε την συνάρτηση ώστε να είναι σε θέση να υπολογίζει το τετράγωνο:

- ενός ακέραιου αλλά και
- ενός αριθμού κινητής υποδιαστολής διπλής ακρίβειας.

Το επιστρεφόμενο αποτέλεσμα θα είναι ανάλογου τύπου με την είσοδο (παράμετρο).

Στη συνέχεια να γράψετε ένα πρόγραμμα το οποίο θα ζητά έναν ακέραιο αριθμό από τον χρήστη, θα τον διαβάξει από το πληκτρολόγιο, θα καλεί την συνάρτηση `square` για να υπολογίσει το τετράγωνό του και τέλος θα εμφανίζει τον αριθμό και το τετράγωνό του στην οθόνη με ανάλογα μηνύματα.

Θα κάνει το ίδιο για έναν κινητής υποδιαστολής διπλής ακρίβειας αριθμό.

Δραστηριότητα 5 – Σύντομες Συναρτήσεις (lambda expressions)

Μεταβείτε στην ιστοσελίδα <https://www.programiz.com/cpp-programming/lambda-expression> και μελετήστε τον τρόπο που μπορούμε να δηλώσουμε και να χρησιμοποιήσουμε μια ανώνυμη συνάρτηση με τη δήλωση lambda expression.

Δείτε το τρόπο με τον οποίο περνάμε παραμέτρους, καθώς και πως μπορεί με τη χρήση του closure να έχει πρόσβαση σε μεταβλητές, που είναι δηλωμένες έξω από τον κώδικα της lambda expression, με τιμή και αναφορά.

Βρείτε τις διαφορές των lambda expressions με τις συναρτήσεις.

Δραστηριότητα 5 – Δημιουργία πρότυπης (template) συνάρτησης

Δημιουργήστε ένα αρχείο με όνομα template_function.cpp με το παρακάτω περιεχόμενο και εκτελέστε το:

```
// overloaded functions
#include <iostream>
using namespace std;

int sum (int a, int b)
{
    return a+b;
}

double sum (double a, double b)
{
    return a+b;
}

int main ()
{
    cout << sum (10,20) << '\n';
    cout << sum (1.0,1.5) << '\n';
    return 0;
}
```

Παρατηρείστε ότι και οι δύο συναρτήσεις sum κάνουν ακριβώς την ίδια ενέργεια: επιστρέφουν το άθροισμα των ορισμάτων που τους δίνουμε όταν καλούνται

Αλλάξτε τον κώδικα του παραδείγματος ώστε να δηλώνετε μια template συνάρτηση ως εξής:

```
// function template
#include <iostream>
using namespace std;

template <class T>
T sum (T a, T b)
{
    T result;
    result = a + b;
    return result;
}

int main () {
    int i=5, j=6, k;
    double f=2.0, g=0.5, h;
    k=sum<int>(i,j);
    h=sum<double>(f,g);
    cout << k << '\n';
    cout << h << '\n';
    return 0;
}
```